

Review

- Define a file pointer named fp.
- Open a file named **data.txt** in **write mode**
- Write character A, B, C to the file
- Close the file after writing.
- Reopen the same file **data.txt** in **read mode**.
- Read from the file, while checking for EOF

Exercise

- Write a program that copies a source text file to a destination file.
 - Both the source and destination file name are command line input
 - Check whether the user input is correct
 - Check whether the source file can be opened
 - Check whether the destination file can be opened

Sample output

```
./copy src.txt dst.txt
```

Exercise

- Write a program that copies a source text file to a destination file.

Steps

1. Initialize
 - Need to use `argc` and `argv[]`
 - Two file pointers, one for reading and the other for writing
2. Check that the input is correct
 - The `argc` should be 3
3. Open the source file in read mode; Check if file exists
4. Open the destination file in write mode

Exercise

- Write a program that copies a source text file to a destination file.

Steps

5. Copy content character by character from source to destination
 - Read the source file using `fgetc [ch = fgetc(sfp) ]` and
 - while the source file pointer is not equal to EOF
 - Write to the destination file pointer using `fputc [fputc(ch, dfp)]`
6. Inform the user and close the files



Files

Random access



File Streams

- Streams provide a channel between files and programs
- There are 3 streams that always exists when program starts:
 - `stdin` - standard input (keyboard)
 - `stdout` - standard output (screen)
 - `stderr` - standard error (screen)
- These names are actually file handlers

Ex: file streams

- `#include<stdio.h>`

```
int main(){  
    char buf[20];  
  
    fscanf(stdin,"%s", buf);  
  
    fprintf(stdout, "%s", buf);  
  
    fprintf(stderr, "%s", "Error");  
  
}
```

Flush



```
int fflush(File *fp)
```

- Flushes (writes) the contents of internal buffer to the file. If successful returns 0 otherwise EOF.
- If a file is opened for updating (reading and writing), there must be an intervening call to “fflush” between a write and read operation.



Random access to file

- `fseek(...)`
 - Seek the pointer position in the file at the specified byte
- `long ftell(fptr)`
 - Returns the value of the current file pointer position
- `void rewind(fptr)`
 - Move the file pointer to beginning of file

fseek()

- `int fseek( FILE *fp, long int disp, int position)`
 - `fp` is the file pointer
 - `disp` is the number of bytes skipped,
 - `-ve` value means move backward
 - `+ve` value means move forward
 - `position` sets pointer position in the file
 - `0 (SEEK_SET)`: beginning of file
 - `1 (SEEK_CUR)`: current position
 - `2 (SEEK_END)`: end of file

Start from the beginning of the file. (e.g., `fseek(fp, 10, SEEK_SET)`; moves to the 11th byte of the file).

Start from the current position. (e.g., `fseek(fp, 5, SEEK_CUR)`; skips 5 bytes forward from where you are now).

Start from the end of the file. (e.g., `fseek(fp, -10, SEEK_END)`; moves to the 10th byte before the end of the file).

Ex: fseek

```
fputs("Hello, world!", fptr);  
fseek(fptr, 7, SEEK_SET);  
fgets(buffer, 20, fptr);  
  
Output: world!
```

- `fseek( p,10L,0)`
 - Skip 10 bytes forward from beginning of file
- `fseek( p,5L,1)`
 - Skip 5 bytes forward from current position of file
- `fseek(p,-5L,2)`
 - Skip 5 bytes backward from end of file

ftell

```
long ftell(FILE *fp);
```

- Returns a long integer as the current position of file pointer
- Returns -1 if an error occurs



Ex: ftell

```
#include <stdio.h>

int main()
{
    FILE* fp = fopen("test.txt", "r");
    char string[20];
    fscanf(fp, "%s", string);
    printf("%s\n", string);
    /* Printing position of file pointer */
    printf("%ld", ftell(fp));
    fclose(fp);
    return 0;
}
```

```
fputs("This is a test.", fptr);
fseek(fptr, -5, SEEK_END);
fileSize = ftell(fptr);
printf("%ld", fileSize);
```

Output: 10

as current position is at 'This is a '

rewind

`void rewind(FILE *fp)`

- Sets the file pointer to the beginning of the file
- Function does not return any value

Ex: fflush

```
#include <stdio.h>

int main() {
    FILE *fp;
    fp = fopen("test.txt", "w+");
    fflush(fp); // Flush the buffer to ensure data is written
    rewind(fp)
    // read file contents
    fclose(fp);
}

return 0;
}
```

Ex:



Write a program to read a text file, data.txt, in reverse order.

- // Open the file in read mode
- // Move the pointer to the end of the file
- // Get the current file position (end position)
- // Move backwards and read each char



Ex: fflush



```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fp;
```

```
    long pos;
```

```
    char ch;
```

```
    // Open the file in read mode
```

```
    fp = fopen("data.txt", "r");
```

```
    // Move the pointer to the end of the file
```

```
    fseek(fp, 0, SEEK_END);
```

```
    // Get the current file position (end position)
```

```
    pos = ftell(fp);
```

```
    printf("Reading file in reverse:\n");
```

```
    // Move backwards and read each char
```

```
    for (long i = pos - 1; i >= 0; i--) {
```

```
        fseek(fp, i, SEEK_SET); // Move  
        pointer to current position
```

```
        ch = fgetc(fp); // Read one char
```

```
        putchar(ch); // Print the char
```

```
    }
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```

